1. 语言模型与 N-gram

语言模型的目标是计算一个句子（词序列）的联合概率：

$$P(x_1, x_2, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_1, \ldots, x_{t-1})$$

为了降低计算难度，引入**马尔可夫假设（Markov Assumption）**，当前词只依赖前 $N-1$ 个词：

- **Bigram (N=2)**：$P(x_t \mid x_1, \ldots, x_{t-1}) \approx P(x_t \mid x_{t-1})$
- **Trigram (N=3)**：$P(x_t \mid x_1, \ldots, x_{t-1}) \approx P(x_t \mid x_{t-1}, x_{t-2})$

参数爆炸与数据稀疏：如果词表大小是 V，N-gram 组合的理论参数数量为 V^N。如果采用拉普拉斯平滑（Laplace Smoothing）解决零概率问题：

$$P_{\text{Laplace}}(x_t \mid x_{t-1}) = \frac{\text{count}(x_{t-1}, x_t) + 1}{\text{count}(x_{t-1}) + V}$$

缺陷：分母直接加上 V 会导致高频词概率被严重稀释，破坏了齐普夫定律的长尾分布。

> 💡 **【Q&A 1】为什么 V^N 会爆炸？拉普拉斯平滑为什么不好？**

> **A**：因为 N-gram 要穷举所有组合。词表 $V = 10000$，3-gram 就有 10^{12} 种组合，存不下。拉普拉斯平滑假设“所有没见过的组合概率相等”，把概率平摊给所有人，直接破坏了原本的语言分布（长尾词极多），导致模型变笨。

2. 齐普夫定律与词表构建

齐普夫定律（Zipf's Law）：词频 n_i 与排名 i 成反比：

$$n_i \propto \frac{1}{i^\alpha}$$

两边取对数，在双对数坐标系下呈一条直线：

$$\log n_i = -\alpha \log i + c$$

第二部分：RNN 的核心机制（记忆的诞生）

1. RNN 的结构与公式

在时间步 t，RNN 接收当前输入 x_t 和上一时刻的隐藏状态 h_{t-1}，更新当前隐藏状态 h_t：

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

如果是语言模型，最终输出预测下一个词的概率分布：

$$\hat{y}_t = \text{softmax}(W_{hy}h_t + b_y)$$

- W_{xh}：输入到隐藏层的权重矩阵。
- W_{hh}：隐藏层到隐藏层的循环权重矩阵（**参数共享**）。
- $\tanh$：激活函数，将值压缩到 $[-1, 1]$ 区间。

> 💡 **【Q&A 3】`num_hiddens`（隐藏单元数）具体长什么样?**

> **A**：它是一个一维数组。假设 `num_hiddens=4`，那么模型每读完一个词，就会吐出类似 `[0.5, -0.1, 0.8, 0.3]` 这样的 4 个数字。这 4 个数字就是模型将历史信息压缩后的“记忆精华”。

> 💡 **【Q&A 4】`len(vocab)` 和 `num_hiddens` 在代码形状里怎么区分?**

> **A**：`len(vocab)` 决定的是**输入**的初始宽度（如 One-hot 编码）和输出层的预测类别数；`num_hiddens` 决定的是 RNN 内部**记忆**的宽度。不管输入多宽，经过 RNN 揉捏后，输出 Y 的最后一维永远是 `num_hiddens`。

第三部分：RNN 的训练、评估与反向传播

1. 损失函数与困惑度（Perplexity, PPL）

总损失为交叉熵：

$$L = \sum_{t=1}^{T} L_t = -\sum_{t=1}^{T} \log P(x_t \mid x_1, \ldots, x_{t-1})$$

困惑度（PPL）：

$$\text{PPL} = \exp\left(-\frac{1}{N}\sum_{t=1}^{N} \log P(x_t \mid x_{<t})\right)$$

2. BPTT（随时间反向传播）

参数共享导致梯度必须沿时间轴累加。以 W_{hh} 为例，梯度展开后存在连乘：

$$\frac{\partial h_t}{\partial h_k} = \prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}}$$

连乘持续 >1 导致梯度爆炸；持续 <1 导致梯度消失。

> 💡 **【Q&A 5】梯度消失有什么严重后果?**

> **A**：模型“记不住”长距离信息。比如：“我小时候在法国长大，所以我的母语是___。” 如果间隔太长，梯度传不到前面，模型提取不出“法国”的特征，就只能瞎猜填“英语”。

3. 梯度裁剪（Gradient Clipping）

当梯度的全局范数超阈值 θ 时，等比例缩小：

$$\text{norm} = \sqrt{\sum_p \sum_i (\nabla_p^{(i)})^2}$$

$$\nabla_p \leftarrow \nabla_p \times \frac{\theta}{\text{norm}}$$

> 💡 **【Q&A 6】梯度裁剪怎么理解?**

> **A**：就像高速限速。最高限速 $\theta = 100$。如果当前车速 `norm=150`，交警不是把你车轮卸了，而是把你整体速度按比例缩小到原来的 $100/150$ （2/3），让车速刚好降到 100。方向（梯度方向）不变，大小被强制缩小。

4. 跨 Batch 的梯度截断（Detach）

处理长文本时，当前 Batch 最终状态传给下个 Batch 作初始状态，但必须切断计算图：

$$h_{\text{init}} \leftarrow \text{detach}(h_{\text{last}})$$

> 💡 **【Q&A 7】为什么要在 batch 之间断开梯度（`detach`）?**

> **A**：不断开的话，计算 Batch 10 的误差时，梯度会顺藤摸瓜一直连回 Batch 1，导致计算图无限长，显存瞬间爆炸。切断后只传数值不传求导链条，每次只更新当前 batch。